Two agencies: equilibria of a monitoring and a management decision

Simon Frost

Table of contents

1 Overview 1

2 Setup 2

3 The joint optimum surveys 4

4 The status quo is an equilibrium 5

5 Who moves first 6

6 What the lenses add 7

7 Summary 7

8 References 8

1 Overview

Monitoring and management are often run by different people. One agency decides whether to survey; another decides what to do on the ground, using whatever the survey reports. Each revises its own decision, against what the other is currently doing. This vignette asks where that settles, using the equilibria of ADR 0020.

An *equilibrium* is a strategy of the whole problem from which no component can gain by changing its own decision alone. It is the solution concept of open games (Ghani et al. 2018), and it is what composes: each component is judged in the context the rest of the chain induces, which the components' lenses carry forward (the boundary law) and backward (the value function). It is not the optimum. An equilibrium can be worth less, and the example below has one: a survey that would pay is never commissioned, because the managers do not use it, and the managers do not use it because it is never commissioned. This is the local optimality of single policy updating

in limited-memory influence diagrams (Lauritzen and Nilsson 2001), with the components as the units that update.

2 Setup

The chain is the zoo's reference habitat network, opened as in the second vignette, with a monitoring component inserted between the soil moisture and the grazing decision.

```
using CategoricalInfluenceDiagrams
using EcologicalBayesianNetworks

habitat = load_model("reference_habitat_bn")
grazing = load_model("reference_grazing_id")
zoo_kernels = Dict{Symbol,FiniteKernel}(x => kernel(habitat, x)
                                        for x in variable_names(syntax(habitat)))
moisture = [:low, :medium, :high]
nothing # hide
```

Five components, each passing on what the next needs:

- site: the abiotic part of the network, generating the soil moisture from climate and irrigation. The soil moisture leaves it hidden.
- monitoring: the decision `Survey` (survey or skip), with no information of its own; a `Reading` of the soil moisture when the survey is made and none otherwise; and the survey's cost. It passes on the soil moisture, still hidden, and the decision and the reading, observed.
- management: the decision `GrazingManagement` (exclude, reduce or maintain), reading the survey decision and the reading; the grazing pressure it achieves; and its cost.
- biotic: the biotic part of the network, from soil moisture and grazing pressure to occupancy.
- valuation: the value of the species' presence, 1000 as in the second vignette.

```
readings = [:low, :medium, :high, :none]
abiotic = bayesnet(:Climate => [:dry, :normal, :wet], :Irrigation => [:low, :high],
                   :SoilMoisture => moisture;
                   mechanisms = [:SoilMoisture => (:Climate, :Irrigation)])
site = Open(InfluenceDiagram(abiotic); outputs = [:SoilMoisture])
monitoring = Open(influence_diagram(:SoilMoisture => moisture, :Survey => [:survey,
:skip],
                                    :Reading => readings;
                                    mechanisms = [:Reading => (:SoilMoisture,
:Survey)],
                                    decisions = [:Survey => Symbol[]],
                                    utilities = [:SurveyCost => :Survey], closed =
false);
                  inputs = [:SoilMoisture], outputs = [:SoilMoisture, :Survey,
:Reading],
                  observed = (outputs = [:Survey, :Reading],))
management = Open(influence_diagram(:SoilMoisture => moisture, :Survey => [:survey,
:skip],
```

```
                                    :Reading ⇒ readings,
                                    :GrazingManagement ⇒ [:exclude, :reduce,
                                    :maintain],
                                    :GrazingPressure ⇒ [:low, :high];
                                    mechanisms = [:GrazingPressure ⇒
:GrazingManagement],
                                    decisions = [:GrazingManagement ⇒ (:Survey,
:Reading)],
                                    utilities = [:ManagementCost ⇒
:GrazingManagement],
                                    closed = false);
                  inputs = [:SoilMoisture, :Survey, :Reading],
                  outputs = [:SoilMoisture, :GrazingPressure],
                  observed = (inputs = [:Survey, :Reading],))
biotic = Open(InfluenceDiagram(bayesnet(:SoilMoisture ⇒ moisture,
                                        :GrazingPressure ⇒ [:low, :high],
                                        :Vegetation ⇒ [:sparse, :moderate, :dense],
                                        :HabitatQuality ⇒ [:poor, :good],
                                        :Occupancy ⇒ [:absent, :present];
                                        mechanisms = [:Vegetation ⇒ (:SoilMoisture,
                                                                     :
                                                                     GrazingPressure),
                                                      :HabitatQuality ⇒
                                                      :Vegetation,
                                                      :Occupancy ⇒
                                                      :HabitatQuality],
                                        closed = false));
              inputs = [:SoilMoisture, :GrazingPressure], outputs = [:Occupancy])
valuation = Open(influence_diagram(:Occupancy ⇒ [:absent, :present];
                                   utilities = [:ConservationBenefit ⇒ :Occupancy],
                                   closed = false);
                 inputs = [:Occupancy])
chain = [site, monitoring, management, biotic, valuation]
whole = foldl(compose, chain)
println(whole)
```

```
OpenInfluenceDiagram(10 variables, 8 mechanisms, 2 decisions, 3 utilities; Symbol[]
-> Symbol[])
```

The survey reads the soil moisture correctly nine times in ten, the two wrong classes sharing the rest, and costs 0.5. Its table is written as a conditional probability table over `(SoilMoisture, Survey, Reading)` and permuted to the kernel's outputs-first layout:

```
reading_cpt = zeros(3, 2, 4)    # (SoilMoisture, Survey, Reading)
for v in 1:3, r in 1:3
    reading_cpt[v, 1, r] = r == v ? 0.9 : 0.05
end
reading_cpt[:, 2, 4] .= 1.0
reading = FiniteKernel(FiniteSpace([FiniteAxis(:SoilMoisture, moisture),
                                    FiniteAxis(:Survey, [:survey, :skip])]),
                       FiniteSpace(:Reading, readings), permutedims(reading_cpt, (3,
                       1, 2)))
```

```
kernels = merge(zoo_kernels, Dict(:GrazingPressure ⇒ kernel(grazing,
:GrazingPressure),
                                  :Reading ⇒ reading))
survey_cost(c) = TabularUtility([FiniteAxis(:Survey, [:survey, :skip])], [-c, 0.0])
presence = FiniteAxis(:Occupancy, [:absent, :present])
utilities = Dict{Symbol,AbstractUtility}(:ManagementCost ⇒
bound_utilities(grazing)[:ManagementCost],
                                         :ConservationBenefit ⇒
                                         TabularUtility([presence],

[0.0, 1000.0]),
                                         :SurveyCost ⇒ survey_cost(0.5))
nothing # hide
```

3 The joint optimum surveys

`optimize` on the closed composite chooses both decisions together. It commissions the survey and acts on the reading:

```
best = optimize(whole, kernels, utilities)
best.expected_utility, only(policy_table(best.strategy[:Survey])),
policy_table(best.strategy[:GrazingManagement])
```

```
(469.08504531250003, :survey, [:reduce :reduce :exclude :maintain; :maintain
:maintain :maintain :reduce])
```

The grazing policy is indexed by the survey decision (rows: survey, skip) and the reading (columns: low, medium, high, none). On the surveyed row it reduces grazing on a low or medium reading and excludes it on a high one; the none column is not reached once the survey is made. Without a survey the best the managers can do is a single action for every site, and reducing grazing is the best of the three:

```
status_quo_strategy(a) = merge_strategies(Strategy(:Survey ⇒
ConstantPolicy(:skip)),
                                          Strategy(:GrazingManagement ⇒
                                          ConstantPolicy(a)))
fixed = [a ⇒ contextual_value(whole, kernels, utilities, status_quo_strategy(a))
         for a in (:exclude, :reduce, :maintain)]
```

```
3-element Vector{Pair{Symbol, Float64}}:
  :exclude => 468.11831874999996
   :reduce => 468.3248000000003
 :maintain => 451.44741875000017
```

The survey is worth about 0.76 more than it costs:

```
status_quo = status_quo_strategy(:reduce)
best.expected_utility - contextual_value(whole, kernels, utilities, status_quo)
```

```
0.7602453124997055
```

4 The status quo is an equilibrium

Each agency's context is what the rest of the chain does with its outputs. `component_contexts` computes it for every component, by the forward pass of boundary laws and the backward pass of value functions. Under the status quo the managers reduce grazing whatever the reading, so the value of the monitoring component's surveyed outputs does not depend on the reading, only on the true soil moisture:

```
ctx = component_contexts(chain, kernels, utilities, status_quo)
states_out = joint_states(codom(interpret(monitoring, kernels, utilities,
                                          Strategy(:Survey ⇒
                                          ConstantPolicy(:skip)))))
[s ⇒ round(v; digits = 3) for (s, v) in zip(states_out, ctx[2].continuation)
 if s[2] === :survey]
```

```
12-element Vector{Pair{Tuple{Symbol, Symbol, Symbol}, Float64}}:
       (:low, :survey, :low) => 357.7
    (:medium, :survey, :low) => 478.7
      (:high, :survey, :low) => 557.9
    (:low, :survey, :medium) => 357.7
 (:medium, :survey, :medium) => 478.7
   (:high, :survey, :medium) => 557.9
      (:low, :survey, :high) => 357.7
   (:medium, :survey, :high) => 478.7
     (:high, :survey, :high) => 557.9
      (:low, :survey, :none) => 357.7
   (:medium, :survey, :none) => 478.7
     (:high, :survey, :none) => 557.9
```

So a survey only adds its cost. And the managers, who never receive a reading, cannot do better than their single best action. Neither agency can gain by changing its own decision: every regret is zero, and the status quo is an equilibrium.

```
regrets(chain, kernels, utilities, status_quo)
```

```
5-element Vector{Float64}:
 0.0
 0.0
 0.0
 0.0
 0.0
```

```
is_equilibrium(chain, kernels, utilities, status_quo),
is_equilibrium(chain, kernels, utilities, best.strategy)
```

```
(true, true)
```

Both the status quo and the optimum are equilibria; the optimum is worth more. Revising one decision at a time from the status quo changes nothing:

```
from_status_quo = best_response_dynamics(chain, kernels, utilities, status_quo)
(values = from_status_quo.values, rounds = from_status_quo.rounds,
 converged = from_status_quo.converged)
```

```
(values = [468.3248000000003], rounds = 1, converged = true)
```

5 Who moves first

Suppose the monitoring agency starts surveying while the managers keep reducing grazing everywhere. Now both could gain: the monitoring agency by dropping a survey nobody uses, the managers by acting on the readings they now receive.

```
surveyed = merge_strategies(Strategy(:Survey ⇒ ConstantPolicy(:survey)),
                            Strategy(:GrazingManagement ⇒ ConstantPolicy(:reduce)))
regrets(chain, kernels, utilities, surveyed)
```

```
5-element Vector{Float64}:
 0.0
 0.5000000000000568
 1.2602453124999897
 0.0
 0.0
```

`best_response_dynamics` visits the components in the order of the chain, so the monitoring agency revises first. It drops the survey, and the managers are back where they started:

```
monitoring_first = best_response_dynamics(chain, kernels, utilities, surveyed)
(values = monitoring_first.values, changed = monitoring_first.changed,
 survey = only(policy_table(monitoring_first.strategy[:Survey])),
 back_to_status_quo = monitoring_first.value ≈ contextual_value(whole, kernels,
utilities,
                                                               status_quo))
```

```
(values = [467.8247999999998, 468.3248000000003], changed = [2], survey = :skip,
back_to_status_quo = true)
```

If the managers revise first, they best respond to the readings in their context, and the survey is then worth keeping. That pair is the joint optimum, and an equilibrium (the largest regret is rounding error):

```
manager_context = component_contexts(chain, kernels, utilities, surveyed)[3]
response = contextual_optimize(management, kernels, utilities;
                               prior = manager_context.prior,
                               continuation = manager_context.continuation)
managers_first = merge_strategies(Strategy(:Survey ⇒ ConstantPolicy(:survey)),
                                  response.strategy)
(value = contextual_value(whole, kernels, utilities, managers_first),
 optimum = best.expected_utility,
 equilibrium = is_equilibrium(chain, kernels, utilities, managers_first),
 largest_regret = maximum(regrets(chain, kernels, utilities, managers_first)))
```

```
(value = 469.0850453125, optimum = 469.08504531250003, equilibrium = true,
largest_regret = 1.1368683772161603e-13)
```

Which equilibrium the two agencies reach depends on where they start and on who revises first. Separate revision cannot be relied on to find the optimum: the information is worth commissioning only to managers who will use it, and only managers who receive it can learn to use it.

6 What the lenses add

The contexts above are computed component by component, by passing laws forward and value functions backward along the chain; the closed composite is needed only for the optimum and for checking. That each component's regret is exactly the gain it would bring to the whole chain is the contextual identity; that the contexts do not depend on how the chain is bracketed is the lens law proved in the Lean module `Lens` (`lens_comp`). The same module proves that a global optimum is always an equilibrium (`seq_sound`) and that the converse fails, on two decisions that must agree with the later one blind to the earlier (`coordination_equilibrium_not_optimal`). The monitoring problem here is that witness at ecological scale: the survey and the response to it must be chosen together.

7 Summary

A monitoring decision and a management decision, held by two components of one chain, have two equilibria: survey and act on the reading, which is the joint optimum, and neither, which is worth about 0.76 less. From the status quo neither agency can gain alone. From a survey nobody yet uses, the outcome depends on who revises first. `optimize` on the closed composite finds the optimum; `component_contexts`, `regrets` and `best_response_dynamics` show what each agency sees, whether it can gain alone, and where separate revision settles.

8 References

Ghani, Neil, Jules Hedges, Viktor Winschel, and Philipp Zahn. 2018. "Compositional Game Theory." *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2018)*, 472–81. https://doi.org/10.1145/3209108.3209165.

Lauritzen, Steffen L., and Dennis Nilsson. 2001. "Representing and Solving Decision Problems with Limited Information." *Management Science* 47 (9): 1235–51. https://doi.org/10.1287/mnsc.47.9.1235.9779.